Pattern Recognition and Machine Learning - Assignment II

Year 2025-2026- Semester II

CCE2502

developed by - Adrian Muscat, 2026

Write your name, ID Card Number below

Dave Galea 0283404L

In this assignment you will be developing Python code to experiment with the error based predictive models.

All questions are graded and total to 100. This assignment contributes to a maximum of 30% of the final study unit score.

In case of any ideas or expected results obtained from literature, cite the proper publication. In general you should not cite blogs (If you do use blogs or AI tools, trace the original document and cite that)

Submit on VLE:

1. This jupyter notebook complete with answers to the questions or tasks (.ipynb)
2. A pdf copy of the same jupyter notebook with all cells executed
3. The completed and signed plagiarism and collusion form

All zipped into one file.

NOTE: before you save your final file, **reset the kernel and execute all the cells**.

NOTE:

This work is to be attempted individually. It is essential that the work you eventually submit and present for your assignment consists only of your own work; use of copied material will be treated as plagiarism or collusion. Discussion is only permitted on general issues, and it is absolutely forbidden to discuss specific details with anyone and/or share results.

Below are links to the Plagiarism and Collusion form:

https://www.um.edu.mt/media/um/docs/faculties/ict/mainfacultyofictfiles/PlagiarismForm.pdf

or at: https://www.um.edu.mt/ict/students/formsguidelines/

In [1]:
```
# Execute this to record library versions installed on your system
import matplotlib
import numpy
import sklearn
import sys

print("Python version : ", sys.version)
print("numpy version : ", numpy.__version__)
print("sklearn version : ", sklearn.__version__)
print("matplotlib version : ", matplotlib.__version__)
```

```
Python version :  3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0]
numpy version :  2.4.3
sklearn version :  1.8.0
matplotlib version :  3.10.8
```

Part I

This part deals with the computation of the directional derivative used in gradient descent. The main gradient descent code and additional helper functions are given. You should have familiarised yourself with these functions during the lab sessions. There are three tasks in part I.

Background

The main gradient descent code and additionalhelper functions are given below.

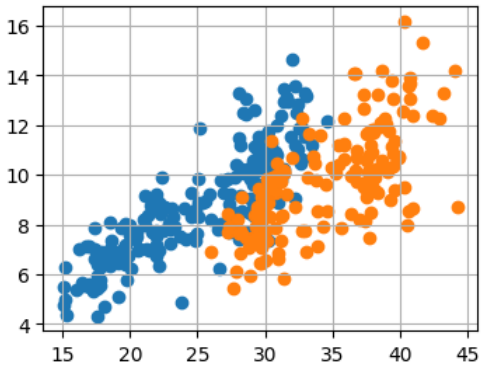
In [2]:
```
# DO NOT MODIFY THIS CELL

import numpy as np
import matplotlib.pyplot as plt
```

```
import csv

# We first load the fish dataset
with open('Fish_Dataset_2020.csv', 'r') as f:
    reader = csv.reader(f)
    data_=[]
    for i,row in enumerate(reader):
        if i==0: print(row)
        else:
            data_.append(row)
data = np.array(data_,dtype=float)
#We take a look at the first 10 examples
data[0:10]
```

```
['Length', ' Width', ' Class(0=spnott', ' 1=awrat)']
```

Out[2]:
```
array([[22.93,  7.24,  1.  ],
       [36.35,  9.97,  0.  ],
       [27.81, 10.4 ,  1.  ],
       [35.84, 11.25,  0.  ],
       [16.53,  5.56,  1.  ],
       [29.76,  8.1 ,  0.  ],
       [17.78,  6.31,  1.  ],
       [29.08,  8.37,  1.  ],
       [19.39,  7.85,  1.  ],
       [35.48, 10.41,  0.  ]])
```

In [3]:
```
# DO NOT MODIFY THIS CELL
# helper function to plot the scatter diagram
def plot_scatter(X, y, display = True):
    """
    X : features  (x1, x2)
    y : binary class  (0/1)
    """
    plt.figure(figsize=(4, 3))
    plt.scatter(X[y==1,0],X[y==1,1])
    plt.scatter(X[y==0,0],X[y==0,1])
    plt.grid()
    if display:
        plt.show()
    return 0

plot_scatter(data[:,0:2], data[:,2])
```



Out[3]: 0

In [4]:
```
# DO NOT MODIFY THIS CELL
# We now split the dataset into the train portion and the test portion
train_size = 0.7
np.random.seed(seed=1) # DO NOT CHANGE THE SEED
np.random.shuffle(data)
train_split = int(len(data)*train_size) # DO NOT CHANGE THE split ratio
print('Dataset size = ',len(data))
print ('Split dataset at ', train_split)
d_train=np.copy(data[0:train_split,:])
d_test = np.array(data[train_split:,:])
print ('Shape of train = ',np.shape(d_train))
print ('Shape of test = ',np.shape(d_test))
```

```
Dataset size =  355
Split dataset at  248
Shape of train =  (248, 3)
Shape of test =  (107, 3)
```

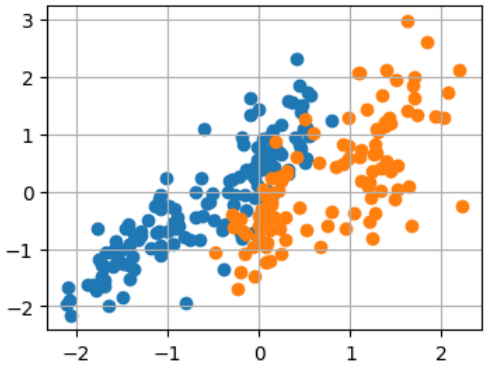
In [ ]:

In [5]:
```
# DO NOT MODIFY THIS CELL
# normalisation
#
X_train_raw = np.copy(d_train[:,0:2])
y_train = np.copy(d_train[:,2])
```

```
X_test_raw = np.copy(d_test[:,0:2])
y_test = np.copy(d_test[:,2])
#
X_mean=X_train_raw.mean(axis=0)
X_var = X_train_raw.var(axis=0)
print(X_mean, X_var)
X_train = np.copy((X_train_raw - X_mean)/np.sqrt(X_var))
print('train mean',X_train.mean(axis=0))
print('train var',X_train.var(axis=0))
X_train.shape
#
X_test = np.copy((X_test_raw - X_mean)/np.sqrt(X_var))
print(X_test.mean(axis=0))
print(X_test.var(axis=0))
print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)
```

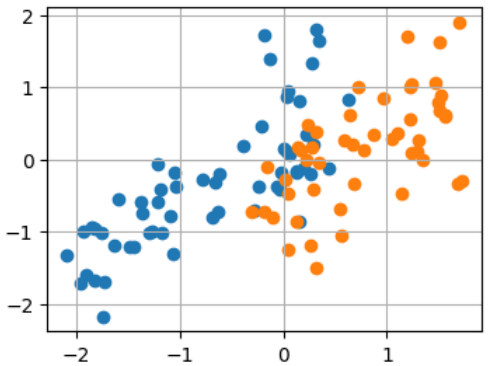
```
[29.26943548  9.31822581] [45.68616904  5.27849847]
train mean [ 3.67688630e-15 -6.71505862e-17]
train var [1. 1.]
[-0.01836152 -0.11731639]
[1.04976307 0.74404694]
(248, 2) (248,)
(107, 2) (107,)
```

In [6]:
```
# plot the scatter diagram for the train set
plot_scatter(X_train, y_train)
```



Out[6]: 0

In [7]:
```
# plot the scatter diagram for the test set
plot_scatter(X_test, y_test)
```



Out[7]: 0

Visually train and test split are similarly distributed and centered at (0,0)

In [8]:
```
# DO NOT MODIFY THIS CELL

def lr_loss_function(y_target,y_predicted):
    """
    inputs:
    y_target: actual target labels (vector of size(m,))
    y_predicted : predicted probabilities for positive class (vector of size (m,))

    outputs
    L:  cross entropy loss
    """
    #
    m = y_target.shape[0]
    # compute the cross-entropy loss

    L = -(y_target*np.log(y_predicted) +
```

```
            (1-y_target)*np.log(1-y_predicted)).sum()/m

    return L

def predict_y_prob(W, D):
    """
    Inputs:
    W : the weight vector [[w_0],[w_1],[w_2],...] of size [N+1,1], where N is number of features, excluding bias
    D : is the feature matrix [m,N] where m is the number of examples

    Outputs:
    y: logits (probability of True Class)
    """

    m = D.shape[0]
    # Compute the probability
    # of the positive class given the weights and features
    # for all m examples and return a vector of size (m,1)

    bias = np.ones([m,1], dtype=float)
    X = np.concatenate((bias,D),axis=1)
    z = np.matmul(W.T,X.T)
    y = 1/(1+np.exp(-z))

    return y.T

def get_accuracy(W, D, y):
    """
    Inputs
    W : the weight vector [[w_0],[w_1],[w_2],...] of size [N+1,1], where N is number of features, excluding bias
    D : the feature matrix [m,N] where m is the number of examples, excluding bias
    y : the target label vector of size (m,)

    Outputs:
    Accuracy (0...1)
    """

    # Compute the accuracy
    # in predicting the positive class and return a scalar

    Y_pred = (predict_y_prob(W, D)>=0.5).astype(int)
    acc = (Y_pred.ravel() == y).astype(int).sum()/y.shape[0]

    return acc
```

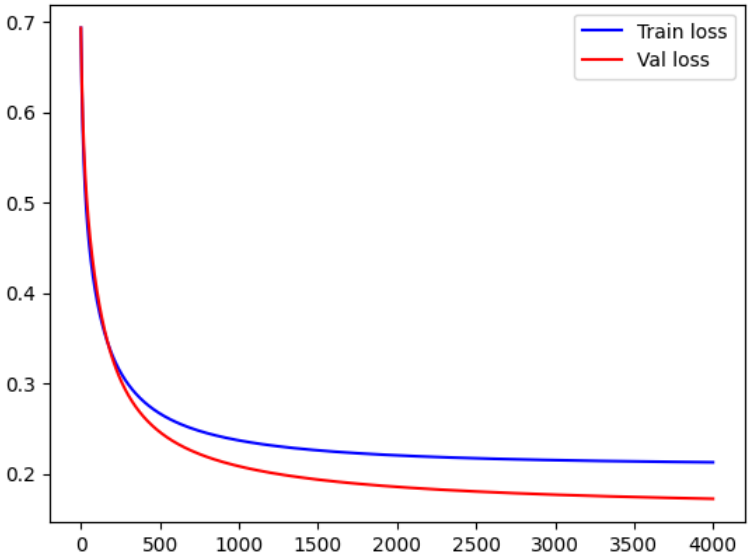
In [9]:

```
# DO NOT MODIFY THIS CELL
# The Gradient Descent algorithm

def lr_train_model(X_data,
                   y_data,
                   split_frac=0.8,
                   init_weight=0.00001,
                   no_iterations=2000,
                   alpha = 0.1):

    # prepare training and validation data
    split = int(y_data.shape[0]*split_frac)
    y_train = np.copy(y_data[0:split])
    y_val = np.copy(y_data[split:])
    X_train = np.copy(X_data[0:split])
    X_val = np.copy(X_data[split:])

    # initialise variables and weights
    N = X_data.shape[1] # number of features
    m = y_train.shape[0] #number of training examples
    W = np.full((N+1,1), init_weight) # initialise weight vectors

    # save loss value per iteration
    history_loss=[]
    history_val_loss=[]

    #  Training loop
    for itr in range(no_iterations):
        y_pred_pr = predict_y_prob(W,X_train).ravel()
        loss = lr_loss_function(y_train,y_pred_pr)
        history_loss.append(loss)
        history_val_loss.append(lr_loss_function(y_val, predict_y_prob(W,X_val).ravel()))
        #

        W[0] = W[0] - alpha*(y_pred_pr - y_train).sum()/m

        for j in range(1,N+1):
            W[j] = W[j] - alpha*((y_pred_pr - y_train)*X_train[:,j-1]).sum()/m

    #
    return {'weights':W, 'history':{'train_loss':history_loss,
```

```
                                        'val_loss':history_val_loss},
                'train_acc':get_accuracy(W,X_train,y_train),
                'val_acc':get_accuracy(W,X_val,y_val)}

def display_results(model):
    #
    history_train_loss = model['history']['train_loss']
    history_val_loss = model['history']['val_loss']
    t=range(len(history_train_loss))
    p1, p2 = plt.plot(t,history_train_loss,'b',t,history_val_loss,'r')
    plt.legend([p1,p2], ['Train loss','Val loss'])
    #
    print('train acc : ', model['train_acc'])
    print('val acc   : ', model['val_acc'])
    #
    print('Weights : ', model['weights'].ravel())
    plt.show()
    return 0
```

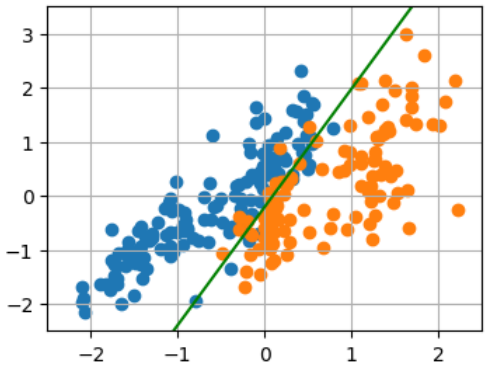
In [10]:
```
# DO NOT MODIFY THIS CELL
lr_fish_model = lr_train_model(X_train, y_train,split_frac=0.8,
                    init_weight=0.00001, no_iterations=4000,
                    alpha = 0.1)
display_results(lr_fish_model)
```

```
train acc :  0.8939393939393939
val acc   :  0.94
Weights :  [ 0.63331016 -6.5782824   3.00937796]
```



Out[10]: 0

In [11]:
```
# DO NOT MODIFY THIS CELL
# We now plot the decision boundary on the scatter plot

plot_scatter(X_train, y_train, display=False)

# Define boundary and plot
cc = -lr_fish_model['weights'][0]/lr_fish_model['weights'][2]
mm = -lr_fish_model['weights'][1]/lr_fish_model['weights'][2]
xx = np.linspace(-2.5, 2.5, 10)
yy = cc + mm*xx
plt.plot(xx, yy,'g')
plt.axis([-2.5, 2.5, -2.5, 3.5])
plt.show()
```



Task I_a

In the first task you will carry out the gradient computation in a separate function and use the returned value in main gradient descent loop. Complete the below analytical_gradient() function and use the result in the gradient descent main loop. The gradient descent function is copied further below with the parts you need to modify marked in between ###...#### and ###...####. Check that you obtain the same output as the output obtained in the background section above.

[10 marks]

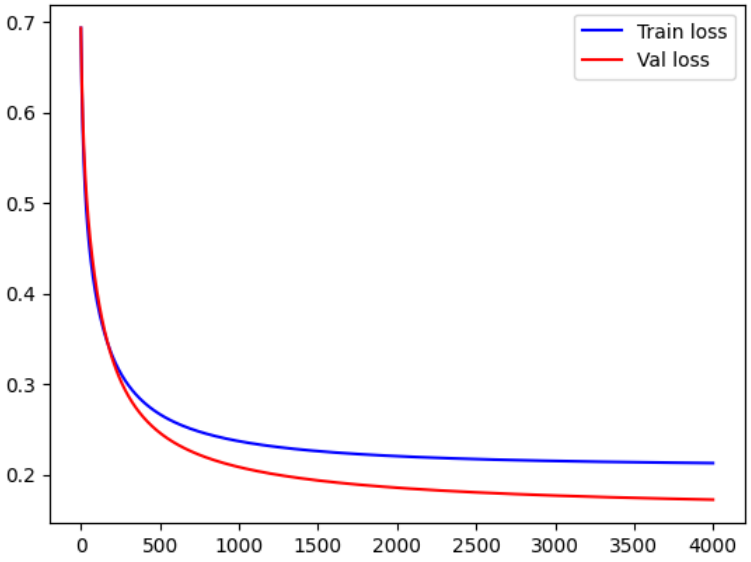
In [12]:
```
# MODIFY THIS CELL (Task I_a)
# Computing the gradients

def analytical_gradient(y_pred, y_target, X):
    """
    inputs:
    y_pred :  predicted probabilities vector of size (m,)  m is number of instances in X
    y_target : target values {0,1}  vector of size (m,)
    X :  input variables (features) used in training loop

    outputs:
    grad : gradients computed analytically, vector of size (N,)

    """
    ################################################################################
    # Write your code below
    m = X.shape[0] # m = number of training samples

    # error term
    error = (y_pred - y_target)

    N = X.shape[1] # number of features
    grad = np.zeros(N+1) # gradient vector is of size features (N) + 1 for bias

    #bias gradient
    grad[0] = error.sum()/m  # average error across all samples

    # Feature gradient
    for j in range(N):
        grad[j+1] = np.sum(error * X[:, j]) /m # average of (error x feature j) across all samples
    ################################################################################

    return grad
```

In [13]:
```
# MODIFY THIS CELL (Task I_a)
# The Gradient Descent algorithm

def lr_train_model(X_data,
                   y_data,
                   split_frac=0.8,
                   init_weight=0.00001,
                   no_iterations=2000,
                   alpha = 0.1):

    # prepare training and validation data
    split = int(y_data.shape[0]*split_frac)
    y_train = np.copy(y_data[0:split])
    y_val = np.copy(y_data[split:])
    X_train = np.copy(X_data[0:split])
    X_val = np.copy(X_data[split:])

    # initialise variables and weights
    N = X_data.shape[1] # number of features
    m = y_train.shape[0] #number of training examples
    W = np.full((N+1,1), init_weight) # initialise weight vectors

    # save loss value per iteration
    history_loss=[]
    history_val_loss=[]

    #  Training loop
    for itr in range(no_iterations):
        y_pred_pr = predict_y_prob(W,X_train).ravel()
        loss = lr_loss_function(y_train,y_pred_pr)
        history_loss.append(loss)
        history_val_loss.append(lr_loss_function(y_val, predict_y_prob(W,X_val).ravel()))
        #

        ################################################################################
        # change code below in task I_a
        grad = analytical_gradient(y_pred_pr, y_train, X_train) # compute full gradient
        W[0] = W[0] - alpha* grad[0] # update bias
```

```
        ##############################################################################
        #



        ##############################################################################
        # change code below in task I_a
        for j in range(1,N+1):
            W[j] = W[j] - alpha*grad[j] # update features
        ##############################################################################

    #
    return {'weights':W, 'history':{'train_loss':history_loss,
                                    'val_loss':history_val_loss},
            'train_acc':get_accuracy(W,X_train,y_train),
            'val_acc':get_accuracy(W,X_val,y_val)}

def display_results(model):
    #
    history_train_loss = model['history']['train_loss']
    history_val_loss = model['history']['val_loss']
    t=range(len(history_train_loss))
    p1, p2 = plt.plot(t,history_train_loss,'b',t,history_val_loss,'r')
    plt.legend([p1,p2], ['Train loss','Val loss'])
    #
    print('train acc : ', model['train_acc'])
    print('val acc   : ', model['val_acc'])
    #
    print('Weights : ', model['weights'].ravel())
    plt.show()
    return 0
```

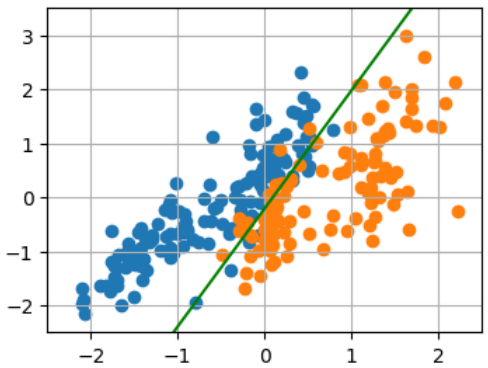
In [14]:
```
# DO NOT MODIFY THIS CELL
lr_fish_model = lr_train_model(X_train, y_train,split_frac=0.8,
                    init_weight=0.00001, no_iterations=4000,
                    alpha = 0.1)
display_results(lr_fish_model)
```

```
train acc :  0.8939393939393939
val acc   :  0.94
Weights :  [ 0.63331016 -6.5782824   3.00937796]
```



Out[14]: 0

In [15]:
```
# DO NOT MODIFY THIS CELL
# We now plot the decision boundary on the scatter plot

plot_scatter(X_train, y_train, display=False)

# Define boundary and plot
cc = -lr_fish_model['weights'][0]/lr_fish_model['weights'][2]
mm = -lr_fish_model['weights'][1]/lr_fish_model['weights'][2]
xx = np.linspace(-2.5, 2.5, 10)
yy = cc + mm*xx
plt.plot(xx, yy,'g')
plt.axis([-2.5, 2.5, -2.5, 3.5])
plt.show()
```



Task I_b

The parial derivatives with respect to each weight will now be approximated using the below forward difference equation. In other words the value of w_j is increased by a small amount (ϵ) to compute the derivative of the loss wrt w_j. The approximation is then used in the gradient descent update equations.

$$\frac{\partial L}{\partial w_j} \approx \frac{L(w_0, \ldots, w_j + \epsilon, \ldots, w_N) - L(w_0, \ldots, w_j, \ldots, w_N)}{\epsilon}$$

1. Complete the function below to obtain a numerical approximation of the gradient and use the numerical approximation inside the gardient descent loop instead of the analytical gradient. (You will need to copy the gradient descent algorithm as modified in task I_a and replace analytical_gradient() function with numerical_gradient() function)
2. Compare the results to those obtained with the analytical gradient computation.

[20 marks]

In [16]:
```
# MODIFY THIS CELL (Task I_b)
# Computing the gradients numerically

def numerical_gradient(y_pred, y_target, X, Weights, eps=0.0001):
    """
    inputs:
    y_pred :  predicted probabilities vector of size (m,)  m is number of instances in X
    y_target : target values {0,1}  vector of size (m,)
    X :  input variables (features) used in training loop
    Weights: vector of size (N+1,1), N is number of features, plus bias
    eps : delta_w to compute gradient

    outputs:
    grad : gradients computed numerically, vector of size (N,)

    """

    ################################################################################
    # Write your code below
    N = X.shape[1] # number of features

    grad = np.zeros(N+1) # gradient vector (size bias + features)

    base_loss = lr_loss_function(y_target, y_pred) # original loss with current predictions

    for j in range (N+1):

        W_copy = np.copy(Weights) # avoiding modifying the original
        W_copy[j] += eps

        y_pred_new = predict_y_prob(W_copy, X).ravel() # new predictions
        new_loss = lr_loss_function(y_target, y_pred_new) # new loss

        grad[j] = (new_loss - base_loss)/eps


    ################################################################################

    return grad
```

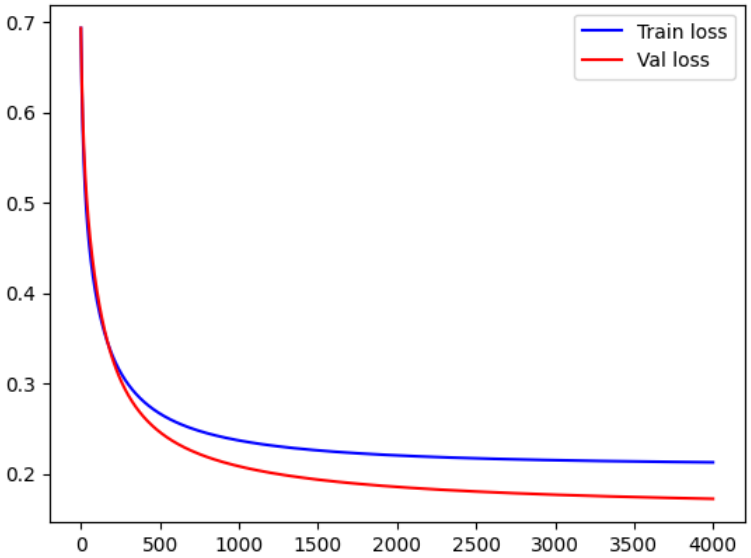
In [17]:
```
# COPY and MODIFY lr_train_model() here
# The Gradient Descent algorithm

def lr_train_model(X_data,
                   y_data,
                   split_frac=0.8,
                   init_weight=0.00001,
                   no_iterations=2000,
```

```
                    alpha = 0.1):

    # prepare training and validation data
    split = int(y_data.shape[0]*split_frac)
    y_train = np.copy(y_data[0:split])
    y_val = np.copy(y_data[split:])
    X_train = np.copy(X_data[0:split])
    X_val = np.copy(X_data[split:])

    # initialise variables and weights
    N = X_data.shape[1] # number of features
    m = y_train.shape[0] #number of training examples
    W = np.full((N+1,1), init_weight) # initialise weight vectors

    # save loss value per iteration
    history_loss=[]
    history_val_loss=[]

    #  Training loop
    for itr in range(no_iterations):
        y_pred_pr = predict_y_prob(W,X_train).ravel()
        loss = lr_loss_function(y_train,y_pred_pr)
        history_loss.append(loss)
        history_val_loss.append(lr_loss_function(y_val, predict_y_prob(W,X_val).ravel()))
        #

        ##############################################################################
        # change code below in task I_a
        grad = numerical_gradient(y_pred_pr, y_train, X_train, W) # compute full gradient (numerical)
        W[0] = W[0] - alpha* grad[0] # update bias
        ##############################################################################
        #



        ##############################################################################
        # change code below in task I_a
        for j in range(1,N+1):
            W[j] = W[j] - alpha*grad[j] # update features
        ##############################################################################

    #
    return {'weights':W, 'history':{'train_loss':history_loss,
                                    'val_loss':history_val_loss},
            'train_acc':get_accuracy(W,X_train,y_train),
            'val_acc':get_accuracy(W,X_val,y_val)}

def display_results(model):
    #
    history_train_loss = model['history']['train_loss']
    history_val_loss = model['history']['val_loss']
    t=range(len(history_train_loss))
    p1, p2 = plt.plot(t,history_train_loss,'b',t,history_val_loss,'r')
    plt.legend([p1,p2], ['Train loss','Val loss'])
    #
    print('train acc : ', model['train_acc'])
    print('val acc   : ', model['val_acc'])
    #
    print('Weights : ', model['weights'].ravel())
    plt.show()
    return 0
```

In [ ]:

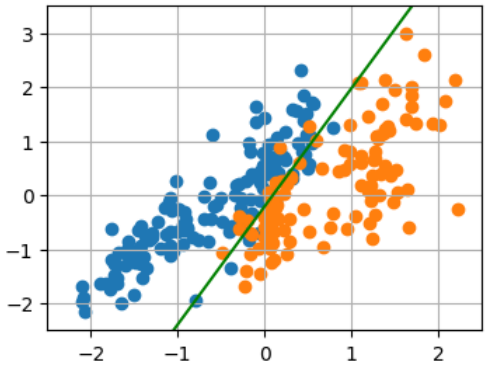
In [18]:
```
# DO NOT MODIFY THIS CELL
lr_fish_model = lr_train_model(X_train, y_train,split_frac=0.8,
                    init_weight=0.00001, no_iterations=4000,
                    alpha = 0.1)
display_results(lr_fish_model)
```

```
train acc :  0.8939393939393939
val acc   :  0.94
Weights :  [ 0.63325767 -6.57816628  3.009289  ]
```



Out[18]: 0

In [19]:
```
# DO NOT MODIFY THIS CELL
# We now plot the decision boundary on the scatter plot

plot_scatter(X_train, y_train, display=False)

# Define boundary and plot
cc = -lr_fish_model['weights'][0]/lr_fish_model['weights'][2]
mm = -lr_fish_model['weights'][1]/lr_fish_model['weights'][2]
xx = np.linspace(-2.5, 2.5, 10)
yy = cc + mm*xx
plt.plot(xx, yy,'g')
plt.axis([-2.5, 2.5, -2.5, 3.5])
plt.show()
```



Part 2: Comparison of results

Both numerical gradient and analytical gradient achieved very similar results, with identical train accuracy, validation accuracy, and very slightly different weights. The difference in weights is due to the numerical gradients' use of ϵ, which may introduce almost negligible discrepancies.

Task I_c

We can consider replacing the sigmoid function with a piecewise linear approximation, known as the HardSigmoid(z) function.

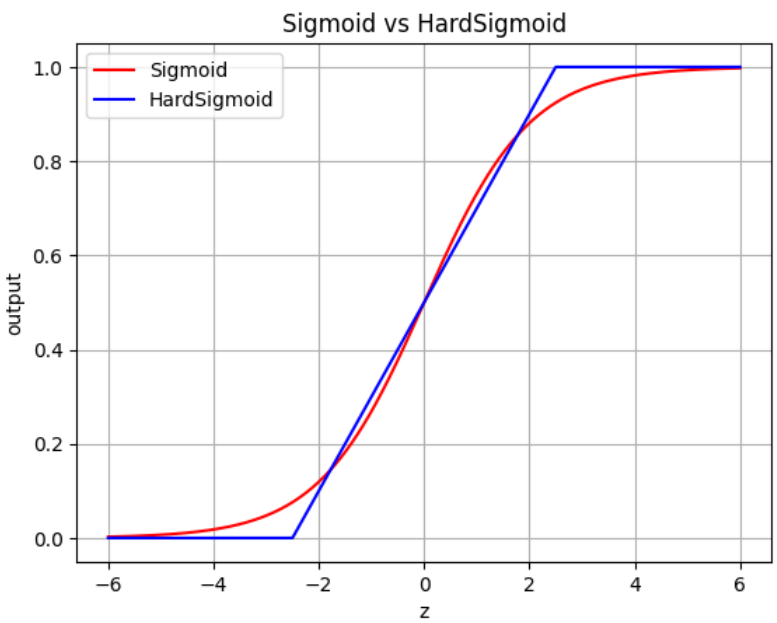
The HardSigmoid(z) function is defined as follows:

$$\text{HardSigmoid}(x) = \begin{cases} 0 & \text{if } x \leq -2.5 \\ 0.2x + 0.5 & \text{if } -2.5 < x < 2.5 \\ 1 & \text{if } x \geq 2.5 \end{cases}$$

1. Plot the HardSigmoid() superimposed on the Sigmoid() (in Python)
2. Assumin we use the HardSigmoid() in our model, explain how the gradients wrt w_j are computed (DO NOT implement in python)
3. Explain the advantages and disadvantages of using a HardSigmoid instead of the standard sigmoid function.

[20 marks]

In [20]:
```
# PLOT here
def hard_sigmoid(z):

    result = np.zeros_like(z)

    for i in range(len(z)):

        if z[i] <= -2.5:
            result[i] = 0

        elif -2.5 < z[i] < 2.5:
            result[i] = 0.2 * z[i] + 0.5

        elif z[i] >= 2.5:
            result[i] = 1

    return result

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-6, 6, 1000) # most graphs i found online went from -6 to 6

plt.figure()
plt.plot(z, sigmoid(z), 'r', label='Sigmoid')
plt.plot(z, hard_sigmoid(z), 'b', label='HardSigmoid')
plt.xlabel('z')
plt.ylabel('output')
plt.title('Sigmoid vs HardSigmoid')
plt.legend()
plt.grid()
plt.show()
```



EXPLAIN HERE

Question 2

When using the HardSigmoid function, gradients are computed using the chain rule, as shown below:

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

- The first term $\frac{\partial L}{\partial \hat{y}}$ comes from differentiating the cross-entropy loss with respect to the prediction, which gives the error: $(\hat{y} - y)$.
- The second term $\frac{\partial \hat{y}}{\partial z}$ is the derivative of the HardSigmoid with respect to z, which is piecewise:

$$f'(z) = \begin{cases} 0 & \text{if } z \leq -2.5 \\ 0.2 & \text{if } -2.5 < z < 2.5 \\ 0 & \text{if } z \geq 2.5 \end{cases}$$

- The third term $\frac{\partial z}{\partial w_j}$ comes from differentiating $z = w_0 + \sum_{j=1}^{N} w_j x_j$ wrt w_j, which simply gives x_j.

Substituting all three terms and averaging over m samples, we get the graident of the loss wrt w_j

$$\frac{\partial L}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) \cdot f'(z^{(i)}) \cdot x_j^{(i)}$$

Question 3

Advantages:

- **Computationally efficient:** avoids the exponential function used in sigmoid [1], and while using fewer resources, it preserves the same properties of the sigmoid function [2]
- **Constant derivative in the linear/activation region:** This prevents the issue of gradients nearing zero at the two ends (gradient saturation) [1]

Disadvantages:

- **Zero gradients outside the linear region**: This causes learning to stop completely for those samples, and lead to useless units during training. In standard sigmoid, even though gradients reach close to zero, the weak signals can still update the parameters, while in the Hard Sigmoid, they are essentially dead zones.
- **Not differentiable at the boundary points**: This can affect gradient-based optimisation which depends on differentiation

References: Task 1

[1] K. Sun, Y. Wang, and H. Zhou, "Enhanced pulmonary nodule detection using a transformer framework with dual fusion and gated mechanism," Complex & Intelligent Systems, vol. 11, no. 9, Jul. 2025, doi: https://doi.org/10.1007/s40747-025-02032-2.

[2] H. Sheng, K. Cheng, X. Jin, X. Jiang, C. Dong, and T. Han, "An efficient deep learning framework for mixed-type wafer map defect pattern recognition," AIP Advances, vol. 14, no. 4, Apr. 2024, doi: https://doi.org/10.1063/5.0190985.

In [ ]:

Part II

It is required to install a Feed-Forward-Fully-Connected Neural Network (FFFC-NN) in a small low-power edge device on which the forward ~~and backward~~ computations are carried out on a CPU (i.e serial). The memory available to store and update the Neural Network weights is 130kBytes. i.e. Accuracy may have to be traded off to fit the model in the memory space available.

Carry out the below tasks using the sk-learn.neural_network module and the given dataset.

In [21]:
```
# DO NOT MODIFY THIS CELL

#Import dataset
import pickle

with open("Task_II_dataset.pkl", "rb") as f:
    X_train, X_test, y_train, y_test = pickle.load(f)

X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

Out[21]:
```
((2400, 10), (2400,), (600, 10), (600,))
```

Task II_a

Explain how you can calculate the amount of memory required to store the weights and compute the output of the neural network. In addition explain how you can measure the memory occupied by the weights and feed-forward computations in your sk-learn model.

[10 marks]

The total memory requirement of a fully connected neural network consists of:

1. Parameter memory (weights and biases)
2. Activation memory (temporary forward-pass computations)

These are then added to get the total memory used

1. Parameter Memory (Weights and Biases)

Each layer contributes:

- **Weights:** $n_{in} \times n_{out}$
- **Biases:** n_{out}
- **Total per layer:** $(n_{in} \times n_{out}) + n_{out}$

Total parameter memory:

$$\text{Memory}_{params} = (\text{number of parameters}) \times 8$$

Since sklearn uses float64 (8 bytes per parameter), the maximum number of parameters under 130 kB is:

$$\frac{130 \times 1024}{8} = 16{,}640 \text{ parameters}$$

Example Architecture: [10 → 100 → 50 → 25]

Layer	Weights	Biases	Total
10 → 100	1,000	100	1,100
100 → 50	5,000	50	5,050
50 → 25	1,250	25	1,275
Total			**7,425**

$$\text{Memory} = 7{,}425 \times 8 = 59{,}400 \text{ bytes} \approx 58.0 \text{ kB}$$

To measure the actual memory occupied by the weights in a trained sklearn model, the coefs_ and intercepts_ attributes are inspected, which store the learned weights and biases as NumPy arrays. An example is shown below:

```
total_params = 0
for weights in best_model.coefs_:
    total_params += weights.size  # number of weights in this matrix
for biases in best_model.intercepts_:
    total_params += biases.size   # number of biases

memory_bytes = total_params * param_size
memory_kb = memory_bytes / 1024
```

---

2. Activation Memory (Feed-Forward Computation)

During inference, computation is **serial**, so only two consecutive layers need to be held in memory at once. Activation memory is approximated as:

$$\text{Memory}_{activations} = \max_i (n_i + n_{i+1}) \times 8$$

where n_i are layer sizes including the input and output layers.

An example of how this is done in code is shown below

```
def compute_activation_memory(config, input_count, output_count, param_size):
    layers = [input_count] + list(config) + [output_count]
    largest_pair = 0
    for i in range(len(layers) - 1):
        pair_size = layers[i] + layers[i + 1]
        if pair_size > largest_pair:
            largest_pair = pair_size
    return largest_pair * param_size

activation_memory_bytes = compute_activation_memory(best_config, input_count, output_count, param_size)
activation_memory_kb = activation_memory_bytes / 1024
```

Task II_b

Train and optimise a neural network that fits in the memory of the edge device and compute the final accuracy on the test set. (You should NOT use the test set at any time during training and optimising)

[30 marks]

In [22]:
```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import numpy as np

# Defining memory
memory_threshold = 130 * 1024
param_size = 8 #8 bytes

# Preprocessng / scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```python
X_test_scaled = scaler.transform(X_test)

input_count = X_train.shape[1] # no. of input features
output_count = len(np.unique(y_train)) # no of classes

# Validation split
X_tr, X_val, y_tr, y_val = train_test_split( X_train_scaled, y_train, test_size=0.2, random_state = 42, stratify = y_train)

# List of configurations
configurations = [
    (25,),
    (50,),
    (100,),
    (64, 32),
    (128, 64),
    (32, 16),
    (128, 80, 32),
    (128, 64, 32),
    (64, 32, 16),
    (32, 16, 8),
    (128, 64, 32, 16)
] # hidden layer sizes

best_model = None
best_config = None
best_accuracy = -np.inf

# train and select best model

results = []

for config in configurations:

    model = MLPClassifier(
        hidden_layer_sizes = config,
        activation='relu',
        max_iter=5000,
        random_state=42
    )

    #train
    model.fit(X_tr, y_tr)

    # validate on validation split
    val_predictions = model.predict(X_val)
    val_accuracy = accuracy_score(y_val, val_predictions)

    results.append((config, val_accuracy))

    # select best based on validation
    if val_accuracy > best_accuracy:
        best_accuracy = val_accuracy
        best_config = config
        best_model = model

print("\nValidation results ==================\n")

for config, acc in results:
    print("Config:", config, "| Accuracy:", acc)

print(f"\nhighest validation accuracy was achieved by {best_config}, this is thus chosen as final model")

train_accuracy = accuracy_score(y_train, best_model.predict(X_train_scaled))
test_accuracy = accuracy_score(y_test, best_model.predict(X_test_scaled))

# Check memory (using example code given in 2a ans)-------

# Total number of parameters
total_params = 0

for weights in best_model.coefs_:
    total_params += weights.size# number of weights in this matrix

for biases in best_model.intercepts_:
    total_params += biases.size # number of biases

# Memory in bytes (float64 = 8 bytes) (weights and biases )
wb_memory_bytes = total_params * param_size
wb_memory_kb = wb_memory_bytes / 1024
# Activation memory
def compute_activation_memory(config, input_count, output_count, param_size):
    layers = [input_count] + list(config) + [output_count]
    largest_pair = 0

    for i in range(len(layers) -1):
        pair_size = layers[i] + layers[i+1]
```

```python
        if pair_size > largest_pair:
            largest_pair = pair_size

    return largest_pair * param_size

# Memory in bytes (activation memory)
activation_memory_bytes = compute_activation_memory(best_config, input_count, output_count, param_size)
activation_memory_kb = activation_memory_bytes / 1024

# Total memory
total_memory_bytes = wb_memory_bytes + activation_memory_bytes
total_memory_kb = total_memory_bytes / 1024


print("\nMemory constraint checking =======================\n")
print("Total parameters:", total_params)

print("Memory for weights and biases (bytes):", wb_memory_bytes)
print("Memory for weights and biases (kB):", wb_memory_kb)

print("\nActivation memory (bytes):", activation_memory_bytes)
print("Activation memory (kB):", activation_memory_kb)

print("\nTotal memory (bytes):", total_memory_bytes)
print("Total memory (kB):", total_memory_kb)

if total_memory_bytes <= memory_threshold:
    print("\nMemory threshold respected")
else:
    print("\nWarning: Memory threshold exceeded!")


# Outcomes of training and selection
print("\nFinal results =================\n")
print(f"Best Configuration: {best_config} (highest validation accuracy)")
print("Train set accuracy: ", train_accuracy)
print("Test set accuracy", test_accuracy)
```

```
Validation results =================

Config: (25,) | Accuracy: 0.86875
Config: (50,) | Accuracy: 0.8770833333333333
Config: (100,) | Accuracy: 0.9041666666666667
Config: (64, 32) | Accuracy: 0.9041666666666667
Config: (128, 64) | Accuracy: 0.9145833333333333
Config: (32, 16) | Accuracy: 0.8895833333333333
Config: (128, 80, 32) | Accuracy: 0.9104166666666667
Config: (128, 64, 32) | Accuracy: 0.91875
Config: (64, 32, 16) | Accuracy: 0.9020833333333333
Config: (32, 16, 8) | Accuracy: 0.8583333333333333
Config: (128, 64, 32, 16) | Accuracy: 0.9083333333333333

highest validation accuracy was achieved by (128, 64, 32), this is thus chosen as final model

Memory constraint checking =======================

Total parameters: 11843
Memory for weights and biases (bytes): 94744
Memory for weights and biases (kB): 92.5234375

Activation memory (bytes): 1536
Activation memory (kB): 1.5

Total memory (bytes): 96280
Total memory (kB): 94.0234375

Memory threshold respected

Final results =================

Best Configuration: (128, 64, 32) (highest validation accuracy)
Train set accuracy:  0.98375
Test set accuracy 0.9
```

Task II_c

Through calculation and measurement, show that your model fits the requirements.

[10 marks]

To fit the requirements, two must be satisfied:

1. **Performance must be acceptable**
2. **Memory threshold must be respected** total memory must not exceed 130 kB

---

1. Model Performance and Selection

Eleven architectures were trained and evaluated on a held-out validation split (80/20).

The configuration **(128, 64, 32)** achieved the highest validation accuracy of **91.88%** and was therefore selected as the final model.

Once selected, the model was evaluated on the full training set and held-out test set:

Split	Accuracy
Train	98.38%
Validation	91.88%
Test	**90.00%**

The model achieves 90% test accuracy, demonstrating strong generalisation to unseen data.

---

2. Memory Calculation and threshold checking

The total memory is the sum of parameter memory and activation memory.

Parameter Memory (Weights & Biases)

The selected architecture has layers: **input(10) → 128 → 64 → 32 → output(3)**

Layer	Weights	Biases	Total Parameters
10 → 128	10 × 128 = 1,280	128	1,408
128 → 64	128 × 64 = 8,192	64	8,256
64 → 32	64 × 32 = 2,048	32	2,080
32 → 3	32 × 3 = 96	3	99
Total			**11,843**

With sklearn using **float64 (8 bytes per parameter)**:

$$\text{Parameter memory} = 11{,}843 \times 8 = 94{,}744 \text{ bytes} = 92.52 \text{ kB}$$

Activation Memory

During inference, only the two largest consecutive layers need to be held in memory simultaneously:

$$\text{Memory}_{activations} = \max_i (n_i + n_{i+1}) \times 8$$

The largest consecutive pair is **128 → 64**, so:

$$\text{Memory}_{activations} = (128 + 64) \times 8 = 192 \times 8 = 1{,}536 \text{ bytes} = 1.50 \text{ kB}$$

Total Memory

$$\text{Total Memory} = 94{,}744 + 1{,}536 = 96{,}280 \text{ bytes} = 94.02 \text{ kB}$$

$$94.02 \text{ kB} \leq 130 \text{ kB}$$

thus, the memory constraint is satisfied.

---

3. Conclusion

The model architecture **(128, 64, 32)** satisfies the 130 kB memory constraint with **35.98 kB to spare**, while achieving a test accuracy of **90%**, confirming it meets both the memory and performance requirements.